TradePostmortem — Complete Build Blueprint

Colosseum Frontier Hackathon | Deadline: May 11, 2026

---

0. THE PITCH IN ONE LINE

> *"You lost the trade. We show you exactly why, when, and how much it cost you."*

---

1. WHAT YOU'RE BUILDING

TradePostmortem is a Solana wallet trade analyzer. A user pastes their wallet address, the app pulls their recent swaps from chain, prices each entry and exit using historical OHLCV data, and outputs a ranked breakdown of their worst trades with specific, actionable diagnoses. Every result is shareable as a card on X.

No smart contract needed. No wallet connection required. Pure data product.

---

2. TECH STACK

```
Frontend:     Next.js 14 (App Router) + Tailwind CSS
Backend:      Next.js API Routes (serverless, no separate backend)
Data Layer:   Helius SDK (swap history) + Birdeye API (historical price)
Hosting:      Vercel (free tier, deploy in 1 command)
Image Gen:    html-to-image or satori (for shareable cards)
Language:     TypeScript throughout
```

Why this stack for vibe coding:

- Next.js API routes mean you never have to set up a separate server
- Vercel deploys automatically on every git push
- Helius SDK gives you parsed swaps in plain English, no raw tx decoding
- Birdeye gives you price AT the exact timestamp of every trade

---

3. API KEYS YOU NEED (get these Day 1)

Service	URL	Free Tier
Helius	helius.dev	100k credits/month free

Service	URL	Free Tier
Birdeye	birdeye.so/api	Free tier available
Vercel	vercel.com	Free for personal

Put all keys in `.env.local`:

```bash
HELIUS_API_KEY=your_key_here
BIRDEYE_API_KEY=your_key_here
```

---

4. PROJECT STRUCTURE

```
tradepostmortem/
├── app/
│   ├── page.tsx                # Landing — wallet input form
│   ├── analyze/
│   │   └── [wallet]/
│   │       └── page.tsx        # Results page
│   └── api/
│       ├── trades/
│       │   └── route.ts        # Fetch + parse swaps from Helius
│       └── price/
│           └── route.ts        # Historical price lookup from Birdeye
├── components/
│   ├── WalletInput.tsx
│   ├── TradeCard.tsx           # Individual trade autopsy card
│   ├── ShareCard.tsx           # Shareable image version
│   └── ScoreBadge.tsx          # Overall score display
├── lib/
│   ├── helius.ts               # Helius SDK wrapper
│   ├── birdeye.ts              # Birdeye API wrapper
│   └── analyzer.ts             # Core scoring logic
├── types/
│   └── index.ts                # TypeScript interfaces
└── .env.local
```

---

5. DATA FLOW (read this carefully)

```
User enters wallet address
```

```
        ↓
GET /api/trades?wallet=ADDRESS
        ↓
Helius: fetch last 50 SWAP transactions for wallet
        ↓
Filter: keep only swaps where tokenOut = SOL or USDC
(these are the "sells" — exit points we can score)
        ↓
For each swap: extract {tokenMint, amountIn, amountOut, timestamp}
        ↓
GET /api/price?mint=TOKEN&timestamp=TX_TIMESTAMP
        ↓
Birdeye: historical_price_unix endpoint returns price at that exact moment
        ↓
Also fetch: current price of that token
        ↓
Analyzer: compute score, diagnosis, counterfactual
        ↓
Render TradeCard components sorted by "damage score"
```

---

6. CORE CODE — COPY AND ADAPT

`lib/helius.ts`

```typescript
```

```typescript
import Helius from 'helius-sdk';

const helius = new Helius(process.env.HELIUS_API_KEY!);

export interface ParsedSwap {
  signature: string;
  timestamp: number;          // unix seconds
  tokenMint: string;          // token you bought or sold
  tokenSymbol: string;
  tokenIn: number;            // amount spent (SOL or USDC)
  tokenOut: number;           // amount received (the token)
  isBuy: boolean;             // true = bought token, false = sold token
  source: string;             // "JUPITER", "RAYDIUM", etc.
}

export async function getWalletSwaps(wallet: string): Promise<ParsedSwap[]> {
  const txs = await helius.enhanced.getTransactionsByAddress({
    address: wallet,
    limit: 100,
    type: 'SWAP' as any,
  });

  const swaps: ParsedSwap[] = [];

  for (const tx of txs) {
    if (!tx.events?.swap) continue;

    const swap = tx.events.swap;
    const timestamp = tx.timestamp;

    // Helius gives us tokenInputs and tokenOutputs
    const nativeInput = swap.nativeInput;
    const nativeOutput = swap.nativeOutput;
    const tokenInputs = swap.tokenInputs || [];
    const tokenOutputs = swap.tokenOutputs || [];

    // Buying a token: SOL/USDC in → token out
    if (nativeInput && tokenOutputs.length > 0) {
      const out = tokenOutputs[0];
      swaps.push({
        signature: tx.signature,
        timestamp,
        tokenMint: out.mint,
        tokenSymbol: out.symbol || 'UNKNOWN',
        tokenIn: nativeInput.amount / 1e9,   // lamports to SOL
        tokenOut: out.rawTokenAmount.tokenAmount,
```

```
        isBuy: true,
        source: tx.source || 'UNKNOWN',
      });
    }

    // Selling a token: token in → SOL/USDC out
    if (nativeOutput && tokenInputs.length > 0) {
      const inp = tokenInputs[0];
      swaps.push({
        signature: tx.signature,
        timestamp,
        tokenMint: inp.mint,
        tokenSymbol: inp.symbol || 'UNKNOWN',
        tokenIn: inp.rawTokenAmount.tokenAmount,
        tokenOut: nativeOutput.amount / 1e9,
        isBuy: false,
        source: tx.source || 'UNKNOWN',
      });
    }
  }

  return swaps.sort((a, b) => b.timestamp - a.timestamp);
}
```

lib/birdeye.ts

```typescript
```

```typescript
const BIRDEYE_KEY = process.env.BIRDEYE_API_KEY!;
const BASE = 'https://public-api.birdeye.so';

// Price of a token at a specific unix timestamp
export async function getHistoricalPrice(
  mint: string,
  unixTimestamp: number
): Promise<number | null> {
  try {
    const res = await fetch(
      `${BASE}/defi/historical_price_unix?address=${mint}&unixtime=${unixTimestamp}`
      {
        headers: {
          'accept': 'application/json',
          'x-chain': 'solana',
          'X-API-KEY': BIRDEYE_KEY,
        },
      }
    );
    const data = await res.json();
    return data?.data?.value ?? null;
  } catch {
    return null;
  }
}

// Current price
export async function getCurrentPrice(mint: string): Promise<number | null> {
  try {
    const res = await fetch(
      `${BASE}/defi/price?address=${mint}`,
      {
        headers: {
          'accept': 'application/json',
          'x-chain': 'solana',
          'X-API-KEY': BIRDEYE_KEY,
        },
      }
    );
    const data = await res.json();
    return data?.data?.value ?? null;
  } catch {
    return null;
  }
}
```

```typescript
// OHLCV for a token (used to find the peak after entry)
export async function getOHLCV(
  mint: string,
  timeFrom: number,
  timeTo: number,
  interval = '15m'
): Promise<Array<{ o: number; h: number; l: number; c: number; unixTime: number }>>
  try {
    const res = await fetch(
      `${BASE}/defi/v3/ohlcv?address=${mint}&type=${interval}&time_from=${timeFrom}&
      {
        headers: {
          'accept': 'application/json',
          'x-chain': 'solana',
          'X-API-KEY': BIRDEYE_KEY,
        },
      }
    );
    const data = await res.json();
    return data?.data?.items ?? [];
  } catch {
    return [];
  }
}
```

lib/analyzer.ts

```typescript
```

```
import { getHistoricalPrice, getOHLCV, getCurrentPrice } from './birdeye';

export type DiagnosisCode =
  | 'BOUGHT_THE_TOP'
  | 'SOLD_THE_BOTTOM'
  | 'PAPER_HANDS'           // sold before the real move
  | 'DIAMOND_HANDS_REKT'    // held through and lost
  | 'GOOD_TRADE'
  | 'UNKNOWN';

export interface TradeAnalysis {
  tokenMint: string;
  tokenSymbol: string;
  entryTimestamp: number;
  exitTimestamp: number | null;
  entryPrice: number | null;
  exitPrice: number | null;
  currentPrice: number | null;
  peakPriceAfterEntry: number | null;
  peakTimestampAfterEntry: number | null;
  solSpent: number;
  solReceived: number | null;
  realizedPnlSOL: number | null;
  unrealizedPnlSOL: number | null;
  peakPnlSOL: number | null;       // what you COULD have made
  leftOnTable: number | null;      // peakPnl - realizedPnl
  diagnosis: DiagnosisCode;
  diagnosisText: string;
  damageScore: number;             // 0-100, higher = worse trade
  entryPercentile: string;         // "you bought in top 20% of the candle"
}

export async function analyzeTrade(
  entrySwap: { tokenMint: string; tokenSymbol: string; tokenIn: number; timestamp: n
  exitSwap: { tokenOut: number; timestamp: number } | null
): Promise<TradeAnalysis> {
  const entryTs = entrySwap.timestamp;
  const exitTs = exitSwap?.timestamp ?? null;

  // Price at entry
  const entryPrice = await getHistoricalPrice(entrySwap.tokenMint, entryTs);

  // Price at exit (if sold)
  const exitPrice = exitTs
    ? await getHistoricalPrice(entrySwap.tokenMint, exitTs)
    : null;
```

```
// Current price (if still holding)
const currentPrice = !exitTs
  ? await getCurrentPrice(entrySwap.tokenMint)
  : null;

// Peak price in 24h after entry
const ohlcv = await getOHLCV(
  entrySwap.tokenMint,
  entryTs,
  exitTs ?? entryTs + 86400  // up to exit or 24h
);

let peakPriceAfterEntry: number | null = null;
let peakTimestampAfterEntry: number | null = null;

if (ohlcv.length > 0) {
  const peak = ohlcv.reduce((best, candle) =>
    candle.h > (best.h ?? 0) ? candle : best
  );
  peakPriceAfterEntry = peak.h;
  peakTimestampAfterEntry = peak.unixTime;
}

// P&L calculations (in SOL terms)
const solSpent = entrySwap.tokenIn;
let realizedPnlSOL: number | null = null;
let unrealizedPnlSOL: number | null = null;
let peakPnlSOL: number | null = null;
let leftOnTable: number | null = null;

if (exitSwap) {
  realizedPnlSOL = exitSwap.tokenOut - solSpent;
}

if (currentPrice && entryPrice) {
  const ratio = currentPrice / entryPrice;
  unrealizedPnlSOL = solSpent * ratio - solSpent;
}

if (peakPriceAfterEntry && entryPrice) {
  const peakRatio = peakPriceAfterEntry / entryPrice;
  peakPnlSOL = solSpent * peakRatio - solSpent;
  if (realizedPnlSOL !== null) {
    leftOnTable = peakPnlSOL - realizedPnlSOL;
  }
}
```

```
// Diagnosis
let diagnosis: DiagnosisCode = 'UNKNOWN';
let diagnosisText = '';
let damageScore = 0;

if (exitSwap && realizedPnlSOL !== null) {
  if (realizedPnlSOL > 0) {
    if (leftOnTable && leftOnTable > realizedPnlSOL * 2) {
      diagnosis = 'PAPER_HANDS';
      diagnosisText = `You made ${realizedPnlSOL.toFixed(2)} SOL but left ${leftOn
      damageScore = Math.min(90, (leftOnTable / solSpent) * 50);
    } else {
      diagnosis = 'GOOD_TRADE';
      diagnosisText = `Solid exit. You captured most of the move.`;
      damageScore = 0;
    }
  } else {
    if (entryPrice && peakPriceAfterEntry && entryPrice > peakPriceAfterEntry * 0.
      diagnosis = 'BOUGHT_THE_TOP';
      diagnosisText = `You entered within 15% of the local peak. The token never r
      damageScore = Math.min(100, Math.abs(realizedPnlSOL / solSpent) * 100);
    } else if (peakPnlSOL && peakPnlSOL > 0 && realizedPnlSOL < 0) {
      diagnosis = 'DIAMOND_HANDS_REKT';
      diagnosisText = `Token peaked ${peakPnlSOL.toFixed(2)} SOL in profit after y
      damageScore = Math.min(100, Math.abs(realizedPnlSOL / solSpent) * 100);
    } else {
      diagnosis = 'SOLD_THE_BOTTOM';
      diagnosisText = `You sold near the local low. Token recovered after your exi
      damageScore = Math.min(100, (leftOnTable ?? Math.abs(realizedPnlSOL)) / solS
    }
  }
} else if (!exitSwap && unrealizedPnlSOL !== null) {
  if (unrealizedPnlSOL < -solSpent * 0.5) {
    diagnosis = 'DIAMOND_HANDS_REKT';
    diagnosisText = `Still holding. Down ${Math.abs(unrealizedPnlSOL).toFixed(2)}
    damageScore = Math.min(100, Math.abs(unrealizedPnlSOL / solSpent) * 100);
  }
}

// Entry percentile (how close to the candle high was your entry?)
let entryPercentile = '';
if (ohlcv.length > 0 && entryPrice) {
  const entryCandle = ohlcv.find(c => Math.abs(c.unixTime - entryTs) < 900);
  if (entryCandle && entryCandle.h > entryCandle.l) {
    const pct = ((entryPrice - entryCandle.l) / (entryCandle.h - entryCandle.l)) *
    entryPercentile = `You bought in the ${pct.toFixed(0)}th percentile of that ca
```

```
    }
  }

  return {
    tokenMint: entrySwap.tokenMint,
    tokenSymbol: entrySwap.tokenSymbol,
    entryTimestamp: entryTs,
    exitTimestamp: exitTs,
    entryPrice,
    exitPrice,
    currentPrice,
    peakPriceAfterEntry,
    peakTimestampAfterEntry,
    solSpent,
    solReceived: exitSwap?.tokenOut ?? null,
    realizedPnlSOL,
    unrealizedPnlSOL,
    peakPnlSOL,
    leftOnTable,
    diagnosis,
    diagnosisText,
    damageScore,
    entryPercentile,
  };
}
```

---

7. API ROUTES

`app/api/trades/route.ts`

```typescript
```

```
import { NextRequest, NextResponse } from 'next/server';
import { getWalletSwaps } from '@/lib/helius';

export async function GET(req: NextRequest) {
  const wallet = req.nextUrl.searchParams.get('wallet');
  if (!wallet) return NextResponse.json({ error: 'wallet required' }, { status: 400

  try {
    const swaps = await getWalletSwaps(wallet);
    return NextResponse.json({ swaps });
  } catch (e) {
    return NextResponse.json({ error: 'Failed to fetch trades' }, { status: 500 });
  }
}
```

app/api/analyze/route.ts

```typescript
```

```
import { NextRequest, NextResponse } from 'next/server';
import { getWalletSwaps } from '@/lib/helius';
import { analyzeTrade } from '@/lib/analyzer';

export async function GET(req: NextRequest) {
  const wallet = req.nextUrl.searchParams.get('wallet');
  if (!wallet) return NextResponse.json({ error: 'wallet required' }, { status: 400

  const swaps = await getWalletSwaps(wallet);

  // Group buys and sells by token mint
  const byToken: Record<string, { buys: typeof swaps; sells: typeof swaps }> = {};
  for (const swap of swaps) {
    if (!byToken[swap.tokenMint]) byToken[swap.tokenMint] = { buys: [], sells: [] };
    if (swap.isBuy) byToken[swap.tokenMint].buys.push(swap);
    else byToken[swap.tokenMint].sells.push(swap);
  }

  const analyses = [];

  for (const [mint, { buys, sells }] of Object.entries(byToken)) {
    for (const buy of buys.slice(0, 3)) {  // limit per token to avoid API overload
      // Find the closest sell after this buy
      const matchedSell = sells
        .filter(s => s.timestamp > buy.timestamp)
        .sort((a, b) => a.timestamp - b.timestamp)[0] ?? null;

      const result = await analyzeTrade(
        { tokenMint: buy.tokenMint, tokenSymbol: buy.tokenSymbol, tokenIn: buy.token
        matchedSell ? { tokenOut: matchedSell.tokenOut, timestamp: matchedSell.times
      );

      analyses.push(result);
    }
  }

  // Sort worst trades first
  analyses.sort((a, b) => b.damageScore - a.damageScore);

  return NextResponse.json({ analyses: analyses.slice(0, 10) });
}
```

8. SHAREABLE CARD COMPONENT

Install: `npm install html-to-image`

```tsx
```

```
// components/ShareCard.tsx
'use client';
import { useRef } from 'react';
import * as htmlToImage from 'html-to-image';
import { TradeAnalysis } from '@/lib/analyzer';

const DIAGNOSIS_COLOR: Record<string, string> = {
  BOUGHT_THE_TOP: '#FF4444',
  SOLD_THE_BOTTOM: '#FF8800',
  PAPER_HANDS: '#FFCC00',
  DIAMOND_HANDS_REKT: '#FF4444',
  GOOD_TRADE: '#00FF88',
  UNKNOWN: '#888888',
};

export function ShareCard({ trade }: { trade: TradeAnalysis }) {
  const ref = useRef<HTMLDivElement>(null);

  const download = async () => {
    if (!ref.current) return;
    const dataUrl = await htmlToImage.toPng(ref.current, { width: 800, height: 420 }
    const link = document.createElement('a');
    link.download = `trade-autopsy-${trade.tokenSymbol}.png`;
    link.href = dataUrl;
    link.click();
  };

  const color = DIAGNOSIS_COLOR[trade.diagnosis] ?? '#888';

  return (
    <div>
      {/* The card that gets exported */}
      <div
        ref={ref}
        style={{
          width: 800,
          height: 420,
          background: '#0A0A0A',
          border: `2px solid ${color}`,
          padding: 40,
          fontFamily: 'monospace',
          color: '#FFFFFF',
          position: 'relative',
        }}
      >
        <div style={{ fontSize: 12, color: '#666', marginBottom: 8 }}>
```

```
          TRADE AUTOPSY — tradepostmortem.xyz
        </div>
        <div style={{ fontSize: 36, fontWeight: 700, color }}>
          {trade.diagnosis.replace(/_/g, ' ')}
        </div>
        <div style={{ fontSize: 20, marginTop: 16, color: '#CCC' }}>
          {trade.tokenSymbol}
        </div>
        <div style={{ fontSize: 15, marginTop: 24, color: '#AAA', maxWidth: 600, lin
          {trade.diagnosisText}
        </div>
        {trade.realizedPnlSOL !== null && (
          <div style={{
            position: 'absolute', right: 40, top: 40,
            fontSize: 48, fontWeight: 900,
            color: trade.realizedPnlSOL >= 0 ? '#00FF88' : '#FF4444',
          }}>
            {trade.realizedPnlSOL >= 0 ? '+' : ''}{trade.realizedPnlSOL.toFixed(2)}
          </div>
        )}
        <div style={{ position: 'absolute', bottom: 40, left: 40, fontSize: 12, colo
          on Solana • powered by Helius + Birdeye
        </div>
      </div>

      <button onClick={download} style={{ marginTop: 12 }}>
        Export Card
      </button>
    </div>
  );
}
```

---

9. 14-DAY SPRINT SCHEDULE

Day	Task
1	Scaffold Next.js, get Helius + Birdeye keys, verify raw swap data coming in for a test wallet
2	Build `lib/helius.ts` — get clean parsed swaps for any wallet
3	Build `lib/birdeye.ts` — historical price + OHLCV confirmed working
4	Build `lib/analyzer.ts` — core scoring + diagnosis logic
5	Wire up `/api/analyze` route. Test end-to-end with real wallets

Day	Task
6	Build landing page with wallet input (clean, dark, no frills)
7	Build results page — list of trade cards sorted by damage score
8	Build ShareCard component — export PNG
9	Edge case handling: unknown tokens, wallets with no swaps, Birdeye returning null
10	Deploy to Vercel, test on 10 real wallets, fix what breaks
11	Polish UI — make the results page genuinely good-looking
12	Record demo video (required for Colosseum submission)
13	Write pitch doc + GitHub README
14	Submit

---

10. COLOSSEUM SUBMISSION CHECKLIST

Required:

- [ ] Public GitHub repo (code must be created during hackathon window — your commits prove this)
- [ ] Video pitch deck (2-3 min, screen + voice, explain the problem, demo the product)
- [ ] Technical demo video (show it working on a real wallet)
- [ ] Weekly update posts (optional but increases visibility — post on X tagging @colosseum)

Your README must cover:

- Problem statement (1 paragraph)
- How it works (data flow)
- Tech stack
- How to run locally
- Live demo link (Vercel URL)

---

11. COMMON FAILURE POINTS (handle these early)

Birdeye returns null for meme tokens Lots of low-cap tokens have no price history. Add a graceful fallback: mark the trade as "Price data unavailable" rather than crashing.

Helius rate limits On the free tier, 100k credits/month. Each Enhanced Transaction call = 1 credit per tx. Fetch max 50 swaps per wallet and cache responses. Do not re-fetch on every page load.

Matching buys to sells is fuzzy Don't try to do perfect FIFO accounting. Match by "closest sell after buy for same token." Good enough for the hackathon.

Timestamp mismatch Helius timestamps are in seconds. Birdeye also expects seconds. Double-check you're not accidentally passing milliseconds.

Tokens with decimals Raw amounts from Helius are in smallest units (like lamports for SOL). Always divide by the correct decimal. For unknown tokens, default to 6 decimals.

---

12. PITCH FRAMING FOR JUDGES

Lead with the pain, not the tech:

> "Every Solana trader has stared at a trade they blew and thought: what actually happened there? Dexscreener tells you the chart. Solscan tells you the tx. Nothing tells you the full autopsy — the entry percentile, the peak you missed, and exactly how much you left on the table. TradePostmortem does that in 10 seconds, on any wallet, no connection required."

Then show it working on a real wallet with a real bad trade. The more visual the damage, the better.

The judges are traders too. They've all been there. Make them feel it.

---

13. EXTENSIONS (if you have time)

These are bonuses, do not build them if they risk missing the deadline:

- **Recurring Mistake Detection:** "You've bought the top 4 times this month"
- **vs Benchmark:** "If you'd held SOL instead, you'd be +X SOL"
- **Public Wall of Ls:** opt-in feed where traders post their worst autopsy cards (community retention)
- **Wallet Score (0-100):** overall trading grade shown on landing after analysis

---

Built for Colosseum Frontier Hackathon | Deadline May 11, 2026